



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

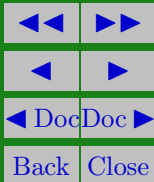
P.O. Box 62000, 00200

Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

ICS 2104 Object-Oriented Programming I

JKUAT SODEL
©2017





ICS 2104 Object-Oriented Programming I

This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 2

The Basics of an Object-Oriented Program Written in C++

Learning outcomes

Upon completing this topic you should be able to:

- Write basic C++ programs
- Explain the importance of comments
- List main sections of a C++ program
- Differentiate between user-defined and programmers' defined classes and functions
- Explain why software are tested.



ICS 2104 Object-Oriented Programming I

2.1. Introduction

C++ programs consists of functions and classes.

Some of these functions and classes are predefined while others are programmers' defined.

It is advisable to use as many pre-defined classes and functions as possible in a program.

Programs that uses pre-defined functions and classes are of better quality than those that are made of programmers's defined classes and functions. Pre-defined functions and classes are stored in header files.

All C++ programs have main() method. This is the first method that is called by the operating system when the program starting executing.

C++ programming language is a case sensitive language, and



ICS 2104 Object-Oriented Programming I

therefore the `main()` function must be written in lower case.

2.2. The basics of a C++ Program

Below are examples of C++ programs.

Example 1: An example of a program that has a function

```
/*=====
```

File name: `First.cpp`

Description: This program demonstrates the minimum number and type of statements necessary to write a simple C++ program.

```
===== */
```

```
#include <iostream.h>
```

```
void print(void);
```



ICS 2104 Object-Oriented Programming I

```
int main(void)
{
    print();
    return 0;
}

void print(void){
    cout<<"Hello world"<<endl;
}
```

- Comments

The above program begins with a comment block.

Comments are used to explain what the program source code is doing.

They make it easier for other programmers especially those



ICS 2104 Object-Oriented Programming I

```
Hello world  
Press any key to continue_
```

Table 2.1: Output from the above program





ICS 2104 Object-Oriented Programming I

that are in charge of maintaining the program to understand it better.

In C++, comments begins with `/*` notations and ends with `*/` notations. Everything between these notations is ignored by the compiler. It is advisable to comment every line of source code.

```
/*This is my first C++ program */
```

One can also use `//(two forward slashes)` notations to comment a line. In that case everything after these notations is ignored by the compiler.

Example:

```
int speed;//store speed of the car
```

- Header Files

The next statement in the program is: `#include<iostream.h>`



ICS 2104 Object-Oriented Programming I

This statement is known as a pre-processor directive. It instructs the pre-processor to retrieve the code stored in `iostream.h` file and insert it at the point of `#include` statements. All files that have `.h` extensions are officially called header files.

- Function Declaration

Function declaration (also referred to as function prototype) provide function interface to the compiler.

The interface of a function consists of following information

1. Return type of function
2. Name of function
3. The number and types of items to be passed to the function.

Return type of function identifies the type of data that a function



ICS 2104 Object-Oriented Programming I

returns to its caller.

For example, a function can return to its caller a value of type `int`. In that case, `int` is the return type of the function.

A function that does not return any data to its caller has `void` as its return type.

The name of function is any meaningful identifier as long as it not a reserved word.

The syntax of a function prototype is:

`return_type_of_function name_of_function(list_of_parameters)`

Note that the function prototype end with a semi-colon.

For example: `int find_area(int length, int width);`

Function prototype can be written with parameter names and without parameter names

For example: Without parameter names: `int find_area(int,`



ICS 2104 Object-Oriented Programming I

int); With parameter names: `int find_area(int length, int width);`
For a function that do not have parameters, void keyword is used in place of parameter list For example: `void find_area(void);`

A function prototype let the compiler get to know the types of parameters, the number of parameters and order of parameters that a function will use. This information enables it to detect type mismatch. Type mismatch is a situation whereby a function is passed values that do not match with the type, the number order of the values that a function expect. For a complex program, it is advisable to have all function prototypes in separate header files. However, for simple programs function prototypes can be included within the body of the program

- `main()` method

The remainder of the program is made of definition of the `main()`



ICS 2104 Object-Oriented Programming I

function. Function definition has two pieces of information: a header and a body. The header provides the function return type, the function name, followed by parentheses.

return type is the type of the value a function returns to its caller. The `main()` function has `int` as its return type. The `int` value that `main()` function returns to the operation system indicates whether the program has executed successfully or not. It can return either zero(0) or one(1). Zero indicate successful execution of the program and one indicate unsuccessful execution of the program.

The `main()` method is written in two forms:

- Form 1

```
int main(void) {  
    ...
```



ICS 2104 Object-Oriented Programming I

```
return 0;  
}
```

- Form 2

```
int main(int argc, char *argc) {  
    ...  
    return 0  
    ;  
}
```

The function body contains the statements to be executed when the function is called. These statements are enclosed in braces ({ and }). In the case of the main() function, it has two statements to be executed when it is called. cout<<“Hello world”<<endl. The first statement print “Hello world” on the screen. The last statement instruct main() function to return an integer (0) to the operating system.



ICS 2104 Object-Oriented Programming I

- Function Definitions

Function definition has the following properties:

1. return type of function: this is the type of the value that the function returns to the caller. If the function does not return any value to the caller, its return is void
2. function name: this is the name of the function
3. list of parameters: this is a list of the type of parameters, the number of parameters and the order of parameters that the function expect from the caller
4. function body: this is a list of the statements that are to be executed when the function is called. The function body is enclosed within opening and closing curly braces ({, }).

Function definition differs from function prototype in the following ways:



ICS 2104 Object-Oriented Programming I

- Function definition must have names of parameters but function prototype must not have names of parameters
- Function definition has function body while function prototype does not have function body

The syntax for a function definition is `return_type_of_function name_of_function(list_of_parameters) { //function body }`

2.3. Testing

The main objective of testing a program is to uncover and remove errors. A successful software test is the one that enables one to uncover and remove errors in a program. An error is a mistake that a program make when writing program source code. There are different types of errors

- syntac errors(also known as compiler errors): Wrong use



ICS 2104 Object-Oriented Programming I

of the syntax rules. These errors are easily detected by the compiler. A program that has syntax errors does not compile

- run-time errors: These are errors that are uncovered when the program is running.
- logical errors: These errors are as a result of wrong logic. They are the most difficult errors to uncover.

2.4. Input and Output Operations

In C++, input and output operations are performed using pre-defined `cout` and `cin` objects. These objects are defined in `iostream.h` standard library. `cout` print strings on the screen while `cin` reads inputs from the keyboard. The general format of `cout` and `cin` calling them is:

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

- `cin>>memory_location1>>memory_location2>>...;`
`cout<<"string";`

`cout` is an object of type `ostream` that links a program to a standard output device (monitor), while `cin` is an object of type `istream` which links a program with a standard input device (keyboard).

The `<<` operator inserts characters into the `cout` for display while the `>>` operator extracts characters from the `cin` object and stores them in memory locations. `>>` operator is referred to as the extraction operator while `<<` operator is referred to as the insertion operator.

Revision Questions

EXERCISE 1.  Explain why comments are important in a program.



ICS 2104 Object-Oriented Programming I

Example . Briefly explain the role of main() method

Solution: The main() method is the one that enables a program to start running. It is the first method that is called by the operating system □

EXERCISE 2.  Explain the importance of header files

EXERCISE 3.  C++ is case sensitive. Explain the phrase “case sensitive”

EXERCISE 4.  Differentiate between syntax errors and semantics

References and Additional Reading Materials

- David Flanagan, 2005, Java in a Nutshell (5th Edition), O'Reilly Press,

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2104 Object-Oriented Programming I

- Matt Weisfeld, 2009, The Object-Oriented Thought Process, 3rd Edition, , Addison-Wesley,
- Malik, D. S. 2002, C++ Programming: From Problem Analysis to Program Design, Course Technology, Canada
- Pappas, H. Chris and Murray, William, H. 1996, C/C++ Programmer's Guide, BPB Publications, New Delhi
- Flanagan, D. (2005). Java in a nutshell : a desktop quick reference. O'Reilly (5th ed.).
- Flanagan, D. (2004). Java examples in a nutshell : a tutorial companion to Java in a nutshell. O'Reilly (3rd ed.)



Solutions to Exercises

Exercise 1. comments improve readability/understandability of a program. They enables other programmers to understand the program better

Exercise 1